

To publish your project cleanly without exposing your credentials, follow this step-by-step setup for **Stage 6: publish to GitHub and write README**.

1. Protect Your Secrets (`.gitignore`)

Create a `.gitignore` file in your root folder immediately. This prevents git from ever tracking your live `.env` secrets.

text

```
# Secrets and environments
.env
.env.local
.env.development.local

# Dependency directories
node_modules/
__pycache__/
*.pyc

# OS metadata
.DS_Store
```

Use code with caution.

2. Share the Blueprint (`.env.example`)

Create a `.env.example` file. This gives peers a dummy template to copy over without leaking your live database credentials.

text

```
SUPABASE_URL=https://supabase.co
SUPABASE_ANON_KEY=your-actual-supabase-anon-or-public-api-key
PORT=3000
```

Use code with caution.

3. Professional Documentation (README.md)

Create a clean, skannable README.md file using the template below:

markdown

Supabase Node/FastAPI Auth Gateway

A fully-featured, production-ready REST API authentication system powered

🚀 Quick Start (Under 5 Minutes)

1. Prerequisites & Environment Setup

Clone this repository to your local machine, then navigate to the root di

```bash

cp .env.example .env

```

Open `.env` and paste your project's `SUPABASE_URL` and `SUPABASE_ANON_KE`

2. Run the Application

Execute the following commands to install dependencies and boot up the mi

For Node.js (Express) lane

npm install

npm run start

For Python (FastAPI) lane

pip install -r requirements.txt

uvicorn main:app --reload

◀ ▶

Use code with caution.

🔧 API Reference Matrix

All protected endpoints expect a standard HTTP authorization bearer token: `Authorization: Bearer <your_jwt_access_token>`.

Route	Method	Access Level	Expected Body / Action	Success Status
-------	--------	--------------	------------------------	----------------

/auth/signup	POST	Public	{"email": "...", "password": "..."}	201 Created
/auth/login	POST	Public	{"email": "...", "password": "..."}	200 OK
/public/info	GET	Public	Unrestricted open lobby text data	200 OK
/auth/logout	POST	Protected	Invalidates active user token session	204 No Content
/protected/profile	GET	Protected	Returns verified safe token metadata	200 OK
/protected/dashboard	GET	Protected	Returns premium dashboard data structures	200 OK

 Interactive Sandbox Documentation (Swagger UI)

Once your application engine fires up, go to `http://localhost:3000/docs` in your browser. Use the native `Authorize` padlock anchor on the upper-right flank to mount your logged-in JWT pass. This lets you execute interactive requests across the network gate dynamically.

```
---

### 4. Git Deployment Protocol
Run these commands sequentially in your terminal console to register your

```bash
1. Double check that .env is safely ignored (should print nothing or sh
git status --ignored

2. Add your clean source files
git add .gitignore .env.example README.md main.js package.json main.py re
```

```
3. Finalize the Stage 6 pipeline checkpoint block
git commit -m "Stage 6: publish to GitHub and write README"

4. Bind your local staging area to your fresh public remote web project
git remote add origin https://github.com
git branch -M main

5. Push your pristine 6-stage development timeline out to the world!
git push -u origin main
```
```

Now that you have built a secure authentication service, what steps should

- * <QuerySuggestion>Implement **Refresh Token matching routines** to seamlessly
- * <QuerySuggestion>Deploy **PostgreSQL Row Level Security (RLS)** scripts
- * <QuerySuggestion>Configure **Docker container configurations** (`Docker

